

ALPS: An Application-Aware, Length-Predictive Scheduler for Efficient LLM Inference at Edge

Chen Chen, Lei Jiao and Richard Mortier

Abstract—The deployment of Large Language Models (LLMs) on resource-constrained edge server clusters introduces a critical systems challenge: achieving high throughput and low latency under strict GPU memory and bandwidth limits. Current schedulers are ill-suited for this environment, as they treat requests independently, incurring two major inefficiencies. They waste compute and memory on redundant prefill computations for identical, lengthy system prompts across applications, and their phase-agnostic policies create suboptimal batches that poorly overlap compute-intensive prefills with memory-bound decodes, leading to GPU under-utilization. This paper presents ALPS, an application-aware, length-predictive scheduler for request serving for edge LLM inference. ALPS transforms the queueing layer into an intelligent orchestrator with two core techniques: (i) *semantic-aware request grouping*, which hashes requests by system prompt to co-locate them, reducing redundant prefill computations; and (ii) *phase-aware scheduling*, which uses a lightweight output token bucket predictor to estimate decode/prefill ratios and strategically reorders requests to maximize prefill-decode overlap in continuous hybrid batching. Augmented with SLO-aware preemption, ALPS is implemented as a practical scheduling layer atop vLLM. Extensive evaluations on an edge GPU testbed with diverse models and production workloads demonstrate that ALPS improves token throughput by up to $1.6\times$ and reduces end-to-end latency by up to $2.4\times$ compared to state-of-the-art schedulers.

Index Terms—Large Language Model, Inference, Edge Computing.

I. INTRODUCTION

LARGE Language Models (LLMs) have exhibited impressive capabilities ranging from reasoning, task planning, and code generation. To achieve the low latency required for interactive applications on desktops and mobile devices, there is a growing imperative to deploy LLMs on edge server clusters closer to users, such as at cellular base stations, Internet exchange points, or enterprise premises [1, 2, 3, 4]. Client software—from GitHub Copilot and ChatGPT to local word processors—increasingly invokes LLMs from these resource-constrained platforms. Edge-cluster LLM serving complements both cloud and personal-device deployment.

This work was partially supported by the European Union’s Horizon Europe research and innovation programme under the Grant Agreement No 101092950 (EDGELESS), and the U.S. National Science Foundation under grants CNS-2047719 and CNS-2225949. (Corresponding author: Richard Mortier.)

Chen Chen is with the Department of Computer Science and Technology, University of Cambridge and the Department of Computer Science, Nottingham Trent University, UK (e-mail: cc2181@cam.ac.uk).

Lei Jiao is with the Center for Cyber Security and Privacy, University of Oregon, USA (e-mail: ljiao2@uoregon.edu).

Richard Mortier is with the Department of Computer Science and Technology, University of Cambridge, UK (e-mail: rmm1002@cam.ac.uk).

Compared with cloud serving, deploying LLMs at the edge reduces communication latency, preserves data locality, improves privacy, and enables operation under intermittent or bandwidth-constrained network conditions [5, 6, 7]. Compared with running an LLM on a single personal device, an edge cluster aggregates the computational resources of multiple edge nodes to support concurrent users, multiple applications, and larger models that cannot be effectively hosted on an individual device. Such deployments are increasingly common in enterprise edge servers, industrial gateways, smart campuses, hospitals, retail stores, and telecom edge infrastructures, where several resource-constrained edge devices cooperate to provide localized AI services. This shift creates a critical system challenge at the edge: the cluster’s LLM serving infrastructure must efficiently schedule and execute a high volume of concurrent requests from diverse applications despite tight hardware constraints, demanding robust solutions to meet stringent throughput and latency requirements for a seamless user experience.

However, edge clusters operate under fundamentally different constraints than cloud data centers. LLM inference for each request consists of two distinct phases: a compute-intensive *prefill* to process the input prompt, followed by a memory-bound *decode* to generate tokens autoregressively [8]. In cloud environments, a common optimization is prefill-decode disaggregation [9, 10, 11], which schedules these phases on separate, specialized hardware. In contrast, edge servers are constrained by limited GPU memory and low inter-GPU bandwidth, making such costly disaggregation impractical [12, 13]. Both phases must be executed on the same device. This necessitates *hybrid batching* [14, 15]—interleaving prefills and decodes within a single GPU, typically via continuous batching with chunked prefill [16, 17]. While the batching engine groups requests for parallel execution, the strategic *queueing* subsystem, which governs request admission, ordering, and prioritization, becomes vital. Its decisions directly control the critical workload mix, balancing new prefills against ongoing decodes fed to the batching engine.

Most existing queuing schedulers treat LLM inference requests as independent, opaque tasks, agnostic to their internal computational structure. This abstraction simplifies design but leads to suboptimal performance in edge environments employing hybrid batching. The scheduler’s blindness directly manifests as GPU under-utilization and elevated tail latency. We identify two specific compounding limitations of this approach as follows:

First, existing schedulers ignore semantic redundancy across requests, particularly in common system prompts from the same applications [18]. Applications often use lengthy, iden-



Fig. 1: Snippet of the prompt structure of three inference requests from Amazon, Gemini, and Claude, respectively [20]. The system prompts (in red) are static and common across users; the user prompts (in green) are dynamic.

tical prompts to define roles, rules, and context, as illustrated in Figure 1. In a hybrid batching system, current schedulers process inference requests with the same system prompt independently, performing redundant prefill computations and allocating duplicate Key-Value (KV) cache entries for identical prefix sequences across requests over different servers. This constitutes a significant waste of precious KV cache space and compute cycles. An efficient scheduler should recognize and co-locate such requests to enable prefix KV cache reuse [19]. This would not only save compute cycles but, more importantly, free up memory to admit more concurrent requests, thereby enhancing the system’s ability to sustain high-throughput, low-latency batch processing. This confirms the substantial design space for avoiding repeated computation of the system prompts.

Second, existing standard queuing policies (e.g., First-Come-First-Serve (FCFS) and similar fairness-oriented approaches as in llm-d [21] and vLLM [16]), though compatible with hybrid batching, ignore requests’ computational profiles and are thus highly inefficient in resource-constrained environments. In a memory-constrained edge system, the long-lived KV cache of a decode-heavy request can block or delay the admission of pending prefill-heavy requests due to lack of available contiguous memory. An FCFS scheduler, unaware of this phase-based resource contention, will not prioritize freeing space to create optimal overlap. Consequently, the system often either misses opportunities for prefill-decode parallelism when complementary requests are not co-located in time, or leaves compute units under-utilized with batches saturated only with memory-bound decodes. This results in lower sustained throughput and higher latency variance compared to a policy that proactively maintains a high-utilization mix of prefills and decodes.

In this paper, to address these limitations, we present ALPS, an application-aware, length-predictive scheduler for request serving on LLM clusters at edge. ALPS is founded on a key insight: the queueing subsystem must evolve from a passive, fairness-oriented dispatcher into an intelligent orchestrator that is semantically aware of request structure and proactively

manages phase composition. To this end, ALPS introduces two novel scheduling primitives: (i) semantic-aware request grouping to exploit system prompt redundancy, and (ii) phase-balanced schedule control to optimize the prefill-decode mix in hybrid batching. By integrating these primitives with a Service Level Objective (SLO) aware preemption mechanism, ALPS unlocks significant performance gains within the strict resource envelope of edge GPUs. We implement ALPS as a practical, non-intrusive scheduling layer atop vLLM [16].

This paper makes the following contributions:

- 1) We identify and quantify static tokens, called system prompts, shared across inputs. Our analysis of production applications shows that such system prompts constitute a significant portion of all the input tokens, presenting a significant, overlooked opportunity for optimization through prefix cache reuse.
- 2) We design and implement ALPS, a novel scheduling framework that introduces application-aware request grouping. ALPS hashes requests by their system prompt to co-locate them in the same request group, enabling global reuse of pre-computed KV caches and reducing redundant prefill work. This transforms prefix sharing from passive, opportunistic cache hits to active, guaranteed scheduling.
- 3) We develop a phase-aware scheduling policy that uses lightweight output length prediction to optimize hybrid batching. ALPS predicts a request’s output token range to estimate its decode/prefill ratio. It then prioritizes request groups with high ratios, strategically reordering the queue to maximize the temporal overlap of compute-bound prefills with memory-bound decodes, sustaining high token throughput.
- 4) We integrate an SLO-aware preemption mechanism that complements our optimizations, maximizing the opportunity to meet strict SLOs without sacrificing token throughput. This provides ALPS with a critical control knob to balance system efficiency with individual request performance.
- 5) We evaluate ALPS extensively on a real edge GPU testbed with diverse production models and workloads. Results show that ALPS improves token throughput by up to $1.6\times$ and reduces end-to-end latency by up to $2.4\times$ compared to state-of-the-art, demonstrating the significant performance headroom unlocked by application- and phase-aware queueing.

Unlike existing LLM serving frameworks that independently optimize scheduling, prefix caching, or SLO management, ALPS introduces a hierarchical scheduling pipeline in which application-aware semantic grouping identifies reusable system prompts, bucket-based output prediction estimates generation characteristics, phase-aware scheduling optimizes hybrid batching at the request-group level, and SLO-aware preemption preserves latency guarantees. The interaction of these components creates capabilities that cannot be achieved by applying the individual techniques independently.

II. BACKGROUND AND MOTIVATION

A. Key-Value Caching and Prefix Sharing

General Technical Background. Efficient LLM inference relies on autoregressive generation, where each output token is produced based on the entire sequence of previous tokens. To avoid recomputing intermediate states, a Key-Value (KV) cache stores the key and value tensors from the LLM’s attention layers for all previously processed tokens in GPU memory. During the decode phase, this cache is iteratively read and updated, reducing computational complexity from quadratic to linear relative to sequence length. *Prefix Sharing* (or *Prefix Caching*) extends this optimization across requests. When multiple input prompts share an identical token sequence at the start, i.e., a common prefix, the system computes and stores the KV cache for this prefix once. Subsequent requests that share this prefix can reuse the cached tensors directly, skipping redundant computation and effectively treating them as immutable states. This optimization is particularly valuable for workloads with repetitive prompt structures, as it saves significant compute cycles and memory bandwidth.

Application	# System tokens	Static
Raycast AI	28	✓
Chatwise	32	✓
Amazon Rufus	172	✓
Claude3	251	✓
DeepSeek	184	✓
Cascade	1916	✓
Wordware	148	✓
GitHub Copilot	1018	✓
Devin	82	✓
Product manager	186	✓

TABLE I: Number of tokens (estimated by OpenAI’s tiktoken [22] using GPT-4o as the baseline) in system prompts per request for diverse applications from ChatgptSystem-Prompt [20] and MetaGPT [23]. Applications include code assistant, reasoning, and planning.

Motivation #1: System Prompts as a Prime Target for Prefix Caching. In real-world LLM serving scenarios, a dominant source of prefix redundancy is the system prompt. LLM applications (e.g., MetaGPT [25], GPTs [20], and Bing Copilot [26]) embed the system prompt, i.e., a special instruction at the start of a conversation, to set the LLM’s context, behaviour, tone, and boundaries. These system prompts often use hundreds of tokens that are static and identical for all users of the same application. As evidenced in our analysis in Table I and Figure 2, system prompts exist widely as fixed static tokens in applications’ inference requests; and in reality, in over 80% of the inference requests, the system prompt is at least half the length of the user prompt in terms of the number of tokens contained, and in over 40%, it is as large or larger. This presents a significant, yet underutilized, opportunity. Current edge serving systems, employing request-agnostic queueing schedulers, process these prompts independently, and requests with the same system prompt cannot be

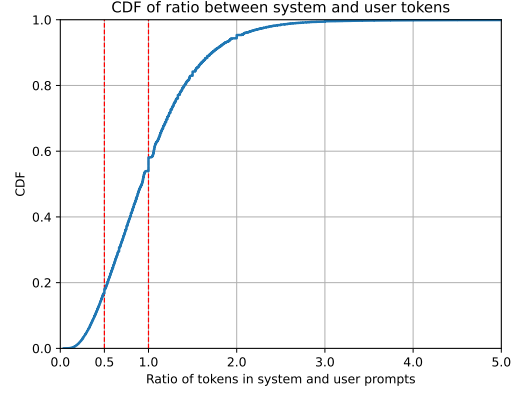


Fig. 2: CDF for the ratio of the number of tokens in the system prompt over that in the user prompt per request in the *llm system prompts benchmark* dataset [24] of 141k requests. System prompts are longer than user prompts for 40% of requests.

aggregated, wasting precious GPU memory and compute on duplicate prefill work. For an edge GPU where memory is the primary bottleneck for concurrency, a scheduler that actively recognizes and aggregates requests that share system prompts can unlock substantial efficiency gains through prefix cache reuse. This directly translates to higher effective throughput and lower latency, forming a core motivation for our proposed scheduler’s semantic-aware design.

B. From Static to Hybrid Batching

General Technical Background. Batching strategies determine how multiple requests are grouped for parallel execution. *Static Batching* processes a fixed set of requests until all complete, leading to poor GPU utilization and high tail latency due to stragglers. *Dynamic Batching* improves this by forming new batches when pending requests accumulate, but can still incur waiting latency. The state-of-the-art is *Continuous Batching* (or iteration-level scheduling), which dynamically updates the batch every decode step, adding new requests and removing finished requests to maximize GPU occupancy [27]. This leads naturally to *Hybrid Batching*, where a single batch contains a mixture of requests in different phases—some undergoing prefill while others are decoding [14]. To mitigate the latency penalty of long prefills and enable finer-grained interleaving, *Chunked Prefill* splits a long prompt’s prefill computation into smaller, fixed-size chunks that can be scheduled between decode steps, improving responsiveness and hardware utilization.

Motivation #2: Request Reordering for Continuous Hybrid Batching. While continuous hybrid batching is necessary for efficiency, its performance on a constrained edge GPU is critically dependent on the composition of each batch. Requests have highly variable prefill/decode ratios (e.g., a short chatbot query vs. a long document summarization). Naive, fairness-based queueing policies (e.g., FCFS) are oblivious to this ratio, leading to suboptimal batch construction. They miss opportunities to overlap a compute-bound prefill with a memory-bound decode from another request, and can create

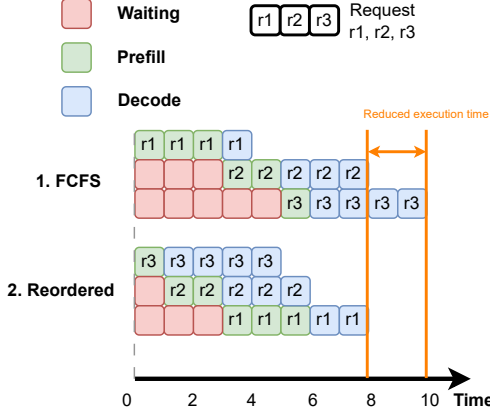


Fig. 3: By considering prefill/decode length, reordering requests can offer better performance than naive First-Come First-Serve (FCFS), given continuous hybrid batching.

batches saturated with only one phase of work, leaving GPU resources underutilized. This creates a clear opportunity: a phase-aware *queueing* policy that strategically reorders or prioritizes requests based on their computational profile. As shown in Figure 3, with continuous hybrid batching equipped, FCFS does not mix the decode phase of $r3$ with the prefill chunks of others; in contrast, reordering $r3$ to the front of the queue can mix its decode phase with other requests’ prefill phases, thus saving total execution time. By controlling the mix of prefills and decodes admitted to execution, such a policy can maximize GPU utilization and minimize latency—another core motivation for our work in designing an intelligent edge scheduler.

III. THE ALPS SCHEDULER

A. Design Principles and Insights

We design ALPS based on three core principles as follows.

- 1) **Application- and Length-Aware Request Grouping.** ALPS ensures that the inference requests that have the same system prompt and (approximately) the same predicted length of the output tokens are aggregated. In this way, the requests are managed at the per-group level instead of the per-request level for multiple benefits. First, the requests from the same application will be more likely to be processed in the same group, maximizing the opportunity to reuse the prefix cache so that LLM instances do not need to recompute from scratch for the redundant system prompts. Grouping also enables shared priority and unified scheduling policies for all requests in the group. Second, ALPS schedules requests without adding significant overhead to compromise throughput and latency. In existing engines such as vLLM, every request entering the scheduler requires sorting, memory planning and batch fitting. By grouping requests, these steps apply once per group instead of once per request, resulting in lower overhead and faster scheduling. In summary, grouping requests with similar

characteristics reduces per-request overhead and avoids compute fragmentation from many unrelated requests.

- 2) **Phase-Aware Request Group Reordering.** ALPS efficiently mixes the prefill phase with the decode phase of different requests via phase-aware reordering of the different request groups. ALPS predicts the range of the output token length for each individual request, calculates the generation ratio for each request, which is defined as the ratio of the output token length over the input token length, and reorders request groups based on the average generation ratio of each group in descending order. By doing so, request groups with large generation ratios are prioritized for execution, providing more chances for the decoded tokens to mix with the prefill chunks of future requests that will arrive. Note that ALPS chooses to predict the range rather than the exact number of the output tokens, since hardware and LLM models can differ, making accurate number prediction difficult, and slight inaccuracy in the predicted number of output tokens does not significantly change the decoding time.
- 3) **SLO-Aware Request Preemption.** While ALPS schedules requests for execution based on the reordered request groups through request fetching, it also estimates the serving time, including the waiting time, the prefill time, and the decode time for each request and prioritizes a request if a potential Service Level Objective (SLO) violation is detected using its estimated serving time. Such request preemption serves as a complementary scheduling mechanism in ALPS, balancing between the phase-aware request mixing for throughput maximization and the individual request’s SLO assurance.

B. System Architecture and Workflow

ALPS is a request scheduler that can be hosted on a master node for the LLM cluster in the edge environment. ALPS consists of multiple components. Physically, ALPS sits between the global request queue (cluster gateway) and the LLM instances in the cluster. An LLM cluster is a computing architecture in which independent LLMs run on separate compute nodes (e.g., servers) and are often connected to one another via local networks. LLM-based applications generate requests that arrive at a global request queue, which serves as the ALPS gateway; the global queue can also be a distributed message broker (e.g., Apache Kafka [28]) that provides scalability and fault tolerance, avoiding a single point of failure. ALPS can be hosted and run either on the same server as the global request queue or on a separate server. An LLM instance is a loaded and running model on a specific node in the cluster.

For a new inference request that arrives at the system, ALPS’ workflow for processing it proceeds as follows, as shown in Figure 4.

First, ALPS either selects an existing request group or creates a new request group for hosting this new request. The *Application Indicator* creates the application ID for the request and stores it in the *Request KV Store*; then, the *Output Token*

Bucket Predictor finds out the token bucket ID by predicting the range of the length of the output tokens for the request and also stores this in the KV store. The two-tuple of the application ID and the token bucket ID uniquely decides the request group for prefix (i.e., system prompt) sharing for this request, where requests in the same request group are executed closely.

Second, ALPS updates the average generation ratio of the request group that is used to host the new request, and reorders the request groups in the same *Serve Queue* to ensure the descending order of the average generation ratio for the corresponding LLM instance. In this order, requests are scheduled group by group for execution on the batching engine. A serve queue is a series of request groups for an LLM instance, and each LLM instance has one and only one serve queue. This reordering operation is initiated and managed by the *Global Scheduler*, which communicates with the *Queue Manager* for concrete queue-manipulation operations. The generation ratio captures the phase-aware reordering strategy of mixing the prefill-phase requests with the decode-phase requests.

Third, to meet the SLO of a request, ALPS uses the *Serving Time Estimator* to estimate the serving time and if a request's expected serving time is likely to violate the pre-set SLO, ALPS schedules this specific request for execution first, regardless of other factors. This is also done by the *Queue Manager*.

In the next three subsections, we elaborate the system components comprising each of the three functionalities, respectively: request grouping, request reordering, and request preemption.

C. Semantic-Aware Grouping & Prefix Sharing

Application Indicator. As LLM applications embed system prompts into all requests from the same application, ALPS groups the requests in terms of the two-tuple, i.e., [Application ID and Token Bucket ID]. The Application ID is created by ALPS' *Application Indicator* component, which assigns a unique ID to each different application and associates this ID with all the requests from that application. As illustrated in Figure 4, ALPS explicitly identifies the application ID. When a request arrives at the global request queue, our Application Indicator first searches the message and identifies keywords such as "role": "system" and "role": "system_prompt", which is defined in the OpenAI chat template [29]. Afterwards, the system content is fetched and hashed into a signed integer using the Python hash function. This integer is then sent to the *Request KV Store*. The KV store checks if the application already exists. If not, a new application ID is created using this integer. The application ID is then inserted to the metadata. The Token Bucket ID is estimated by ALPS' *Output Token Bucket Predictor* component, which will be elaborated next. A request can be further associated with other metadata such as the LLM model type, SLO, and the timestamp.

Serve Queues. ALPS maintains *Serve Queues*, where each serve queue corresponds to one LLM instance and each LLM instance corresponds to one serve queue, as illustrated in Figure 5. Every request includes a model type that specifies

the model selected by the user. Each request, therefore, will be routed to the corresponding instance. If there are multiple instances with the same LLM model, strategies like Round-Robin, Random, least-recently used and other load balancing strategies can be used (which is configurable). In other words, requests are not pinned to one instance. As a new request group is created, it is linked to a serve queue. By checking the model type, each request group knows which serve queue it belongs to; and a serve queue corresponds to an LLM instance. The serve queues are separate from the global request queue, and the latter is the first destination as the external inference requests arrive at the system and is outside ALPS. Within ALPS, we use the serve queue as an abstraction for request popping, request insertion, and request reordering. The order of the request groups in the serve queue represents the execution order of the request groups at the corresponding LLM instance. Using this abstraction, we manage to schedule requests at the request group level, rather than at the individual request level. The serve queues are manipulated by the *Queue Manager* and steered by the *Global Scheduler*, which will be described next.

D. Phase-Aware Scheduling with Length Prediction

Output Token Bucket (OTB) Predictor. After a request is processed by the application indicator, the request information is sent to the *Output Token Bucket (OTB) Predictor*. The OTB is built on top of the distilbert-base-uncased model [30], which was trained for sequence classification to classify which length bucket the output length falls into. DistilBERT is a Transformer model, smaller and faster than BERT [31], which was pretrained using the BERT base model as a teacher. The model learns the same inner representation as its teacher model, but is faster for inference or downstream tasks.

Instead of predicting the number of tokens for each request, OTB predicts the range of the number of output tokens for each request, and each request is classified into a token bucket. For instance, we predict buckets with a bucket size of 50 tokens, so the buckets are [0, 50] tokens, [50, 100] tokens, [100, 150] tokens and so forth. By this means, the number of classifications is significantly lower. The bucket size is configurable to adapt to different hardware and inference models if needed. In other words, if a request is predicted to output 70 tokens, it falls into Bucket ID 2. The prediction is a classification problem by dividing output length into buckets and training the predictor to select the correct bucket for a prompt. The predicted bucket ID is finally inserted to the request metadata so the serve time estimator and the global scheduler can read it.

Our observation is that slight inaccuracy in the predicted number of output tokens does not significantly change the decoding time. The decoding time of each token is less than 0.01 seconds in our experiments, so if the predicted token length is more than the actual token length by 10, the variation is 0.1 seconds, which will not massively change the effect of mixing prefill and decoding. The training details can be found in Section IV-A.

Request KV Store. The request KV store is a key-value store as our storage system which collects request metadata

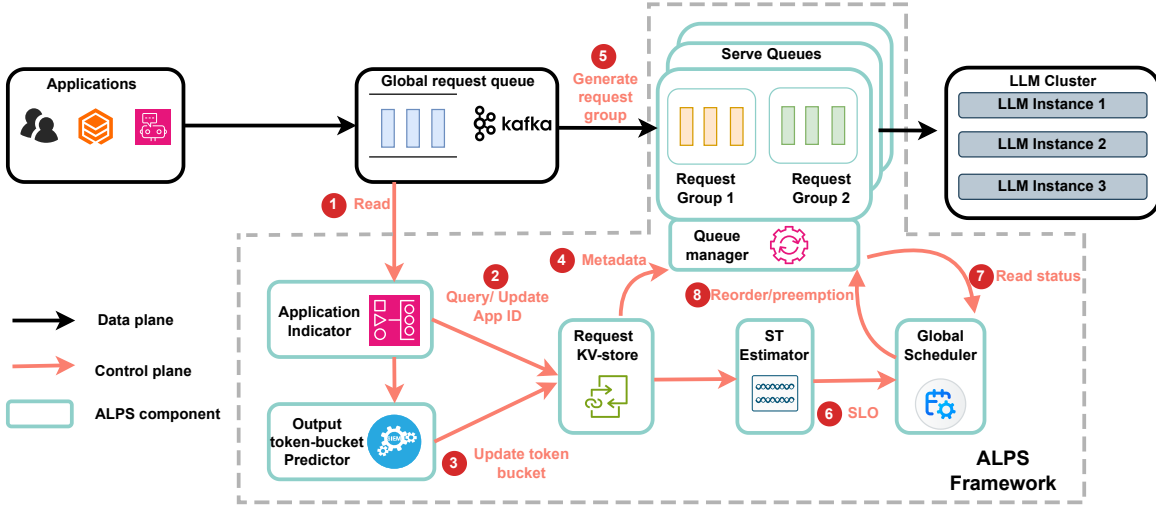


Fig. 4: System architecture of ALPS. The life cycle of a request is as follows: 1. Create an application ID by hashing the system prompt; 2. Store data in the request KV store; 3. Predict the token bucket ID (the range of output token length); 4. Pass metadata to the queue manager; 5. Generate request groups based on the application ID and the token bucket ID. ALPS also periodically executes the following actions: 6. Check potential SLO violation; 7. Read status from the queue manager; 8. Reorder request groups or conduct request preemption.

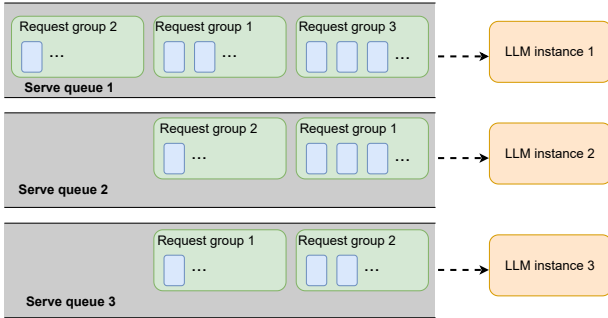


Fig. 5: Visualization of request groups and serve queues.

(application ID and the bucket ID) from the application indicator and the OTB predictor and links the metadata to the request ID. The metadata of requests is then sent to the Queue Manager to create request groups and serve queues. Our global scheduler also retrieves metadata to schedule requests.

Queue Manager: Fetching. The queue manager is an agent that creates request groups and serves queues based on request metadata (application ID, SLOs and required LLM model). Also, it accepts queries and scheduling orders from the request KV store and the global scheduler.

Request Fetching. Request fetching refers to the operation that dequeues requests from the serve queue using a pull-based model and adds them to the running batch. Whenever the total tokens in the running batch fall below the GPU’s capacity, a fetch signal is issued to retrieve a request from the global queue. Note that each request has a *model type* in the metadata, so the request will be sent to the instance with the required LLM model. The specific request fetched is determined by the request group at the head of the serve queue. Within that head group, requests follow a first-come, first-served (FCFS)

order, so the first request to join the group is also the first to be dequeued and added to the GPU’s running batch. Note that request fetching alone is insufficient to immediately serve a request with a low SLO value (i.e., a stricter SLO). A fetch operation can only occur when spare token capacity exists on the GPU device, meaning strict-SLO requests cannot bypass head-of-line blocking.

Global Scheduler. When the queue manager creates a new request group, the global scheduler is triggered to reorder the serve queue. The global scheduler will reorder the request groups in a descending order of the average *Generation Ratio*. The generation ratio of a request refers to the ratio of the output token length over the input token length for the request, formulated as

$$\epsilon = \frac{L_{decode}}{L_{prefill}}$$

Requests with larger ϵ are prioritized, allowing those with longer decoding steps to be processed first. This increases the chance for the decoding tokens to better mix with the prefill chunks and effectively expands the token batch size (see Figure 3). Since the exact number of output tokens is unknown prior to execution, we use the OTB predictor to estimate this.

E. SLO Attainment via Preemptive Scheduling

Serving Time (ST) Estimator. Service Level Objective (SLO) is crucial to user experience. It constrains the first response time of requests and the time interval between tokens [32]. Therefore, we have focused on the time-to-first-token (TTFT) and time-between-token (TBT) in this work. When the SLO of a request is likely to be violated, the global scheduler uses preemptive scheduling to dispatch that specific request first. The core is how to estimate the total serving

time of a request and now we explain how the serving time is estimated by the ST Estimator.

The *Serving Time (ST) Estimator* predicts the total serving time of a request based on its token bucket derived from the OTB predictor. As the global scheduler reorders request groups in the serve queue, some requests may wait a long time in the queue which may lead to SLO violation. If the estimated serving time exceeds the SLO, ALPS will prioritize these requests and dispatch them first.

The total serving time consists of three parts, i.e., waiting time, prefill time and decode time for a request r .

$$T_r = W_r + P_r + D_r$$

The current waiting time W_r can be derived from the request timestamp and the current timestamp. The estimated prefill time per token is nearly constant for each LLM model compared to the decode time. Our experiments show that the prefill time is 50-100 times less than decode time (see our Figure 14b) because it is highly parallel. Therefore, we can estimate the total prefill time by using the product of prefill time per token and the number of input tokens. The prefill time per token is profiled offline for different models. Therefore, the prefill time P_r for a request r is:

$$P_r = I_r \cdot p_x$$

where I_r is the number of input tokens and p_x is the prefill time per token for LLM model x . The estimated decode time can be represented by

$$D_r = O_r \cdot d_x$$

where O_r denotes the number of output tokens which can be derived from the OTB predictor. We have used the upper bound of a token bucket to represent the corresponding token length. For example, if a request is labeled as Bucket 3 [100, 150], we then use 150 as the value of O_r . By this means, we maintain the ST estimator conservative because the LLM inference system cannot guarantee the ideal performance due to the limitation of memory capacity. Some requests are inevitably halted when the GPU memory is exceeded. d_x represents the decode time per output token for LLM model x which is profiled from our tests for each model. In summary, we use the estimated serving time to detect potential SLO violations.

Queue Manager: Request Preemption. Since request fetching alone cannot guarantee immediate service for requests that face SLO violation, ALPS introduces a complementary mechanism called request preemption. Request preemption is initiated when the ST estimator anticipates that a request waits in the queue and is likely to violate its SLO. In response, the global scheduler promotes that request to the front of the serve queue. When preemption occurs, the newly promoted head request is fetched into the running batch as capacity allows.

IV. IMPLEMENTATION AND EVALUATION

We have implemented ALPS using 1200 lines of Python codes on top of the open-source vLLM [16] V1 engine, where chunked prefills and continuous hybrid batching are supported.

A. Experiment Setup

We evaluate ALPS over several open-source models, including Google Gemma-3, Meta Llama-3 and Huggingface SmolLM families. We selected these models to make sure they can be hosted on our edge devices. We have used different batch sizes ranging from 8 to 64. We have used the vLLM engine to invoke these models, where the length of generated tokens is 4096. The size of the chunked prefill ranges between 512 and 2048. Our evaluation aims to validate the effectiveness of ALPS's scheduling strategy under a representative edge-cluster deployment rather than to characterize distributed communication scalability or network bottlenecks.

Testbed. We set up a testbed of 4 NVIDIA Jetson AGX Orin with 2048-core Ampere GPU, 64 GB memory, and 64 Tensor cores. One Orin node hosts ALPS as a master node, and three other nodes use vLLM and work as worker nodes. The CPU is ARMv8@2.2GHz with 8 cores. The operating system is Ubuntu 20.04. The AGX Orin is published as Edge AI device by Nvidia and therefore we select it for our testbed. To imitate the resource bottleneck in the edge environment, we have limited the maximum memory for each LLM instance to 19.2GB by using the parameter *gpu_memory_utilization* in vLLM. The bandwidth between two nodes is 6 Gbps, tested by iperf3 [33].

To show that ALPS performs well for different LLM models, we have conducted two sets of experiments: (1) **Same-Models** refers to deploying the same model (Gemma-3-1B) on the three nodes in the cluster, where each node hosts a model. (2) **Mixed-Models** refers to three different models (Gemma-3-1B, Llama-3-1B and SmolLM2-360M) in the cluster, where each node hosts a model.

Baselines. To evaluate the performance of ALPS, we consider the following three baselines: (1) **QLM** [34] groups requests based on model and SLOs, prioritizes requests with urgent SLOs. Similar to ALPS, QLM is also a cluster-level scheduler that manages LLM request queues. (2) **vLLM+app** combines our application indicator with vLLM engine, grouping requests based on their applications but not using the reordering technique. (3) **vLLM** [16] uses the first-come-first-serve scheduler in vLLM engine.

Type	Dataset	Request Size
Conversation	AlpacaEval [35]	805
Question Q&A	MS MARCO [36]	102k
Code generation	NI2bash [37]	609
Math problem	Orca-math [15]	200k
Finance	Finance-alpaca [38]	68.9k

TABLE II: Evaluation datasets.

Workloads. Our evaluations are performed to execute ten representative LLM applications, i.e., Research Assistant, Customer Service Representative, Composing Functions, Sales Assistant, Product Documentation, Smart Assistant, Engineer, Teacher, QA Engineer and Product Manager. The applications

are derived from MetaGPT [23], which provides a system architect to design APIs. For system prompts, we use the datasets from PromptCraft [20]. The user prompts are derived from five real-world datasets as given in Table II. The type of prompts includes conversion, Question and Answers, Code Generation, Math problem and Finance. Unless, it is particularly specified, the number of inference requests is 1000.

OTB Predictor Training. We have implemented the OTB predictor by fine-tuning the *distilbert-base-uncased* model [30]. We train our output token bucket predictor using the Alpaca dataset [39] with 52,002 requests. We first categorize each request into a bucket based on the number of output tokens in the response. Then, we use these buckets as labels and the instruction prompts as the inputs to fine-tune the *distilbert-base-uncased* model, predicting the range of output tokens for new requests.

We achieve an accuracy of 79.12% with a bucket size of 50 tokens by dividing the datasets into 80% as the training set and 20% as the test set. We train the model for 5 epochs with a learning rate of $2e^{-5}$. Finally, we load the model weights and use the trained model for prediction. As OTB only predicts the range of output token numbers, it generalizes well to other datasets used in the experiment. We also present a study on the bucket size in the following section (Section IV-D).

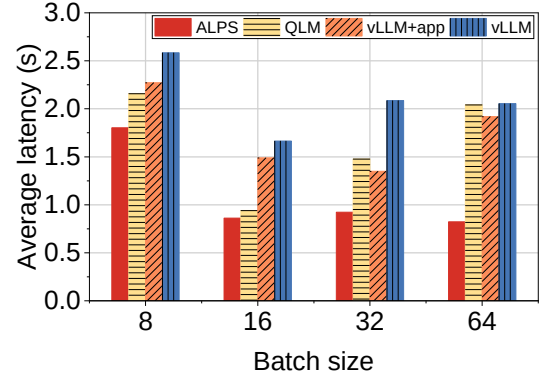
Performance Metrics. There are three primary latency metrics of interest for LLM serving: End-to-End (E2E) latency perceived by users, Time-To-First-Token (TTFT) and Time-Between-Tokens (TBT). In this work, for a given request, end-to-end latency refers to the total time from when a request is made (i.e., user sends a prompt) to when the final token of the response is received. E2E latency is a key metric for interactive use cases (e.g., chatbots), because users care about how long the entire response takes to arrive. TTFT measures the time of generating the first output token from the time a request arrives in the system. This metric is important as it reflects the initial responsiveness of a system. TBT measures the time interval between the generation of tokens, which impacts the overall perceived fluidity of the response.

SLO Attainment. A request that satisfies its Service Level Objective (SLO) can be considered operationally effective. Consequently, our evaluations primarily focus on comparing the throughput of LLM service frameworks using different scheduling mechanisms under SLO-compliant conditions. In the following experiments, the SLO thresholds are set to an end-to-end latency of less than 5~10 seconds and the time-between-token less than 0.1~0.2 seconds according to previous study [17, 32].

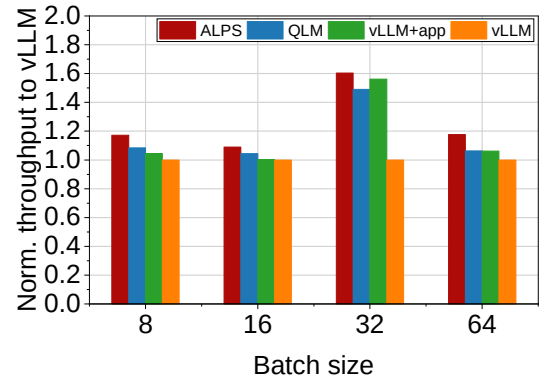
Figure 7 illustrates the Cumulative Distribution Function (CDF) of different methods for end-to-end latency. ALPS consistently outperforms other baselines, with approximately 95% of the requests completing within 5 seconds, compared to 7.5 seconds for other baselines. The largest improvement is observed in the upper tail, where ALPS substantially reduces the tail latency.

B. Same-Models Evaluations

We run ALPS on 3 Nvidia Orin devices with 3 instances of Gemma-3-1B-it to evaluate the same-models serving perfor-



(a) E2E Latency in seconds



(b) Normalized Throughput to vLLM

Fig. 6: Performance on three Gemma-3-1B models in a cluster of 3 Nvidia Orin with different batch sizes.

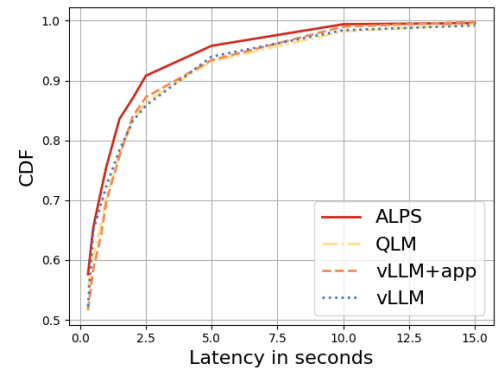


Fig. 7: CDF of latency in a cluster of 3 Nvidia Orin with three Gemma-3-1B models.

mance regarding request latency, throughput, TTFT and TBT.

E2E Latency and Throughput in Same-Models Cluster. Figure 6 shows the request latency and system throughput achieved by ALPS and baselines. The E2E latency is the end-to-end latency perceived by the users. We observe that ALPS receives the best performance in terms of E2E latency over varying batch sizes. This stems from two aspects: (1) ALPS groups requests based on their system prompts, and efficiently reuses the prefix-caching mechanism. (2) ALPS reorders request based on the estimated output token length,

and mixes the prefill and decode phases between requests. As the batch size increases, we find that the latency of ALPS fluctuates between 0.82 to 1.8 seconds, while that of baselines ranges from 0.9 to 2.6 seconds. Compared to QLM, which uses profiling to estimate the request waiting time, ALPS predicts the ranges of output length and leverages the concept of "mixed batching" to fully unleash the potential of hardware. Compared to vLLM, ALPS is able to group requests with same system prompt and reuses prefix caching.

For token throughput in Figure 6b, we normalized all the throughput to that of vLLM. ALPS obtains the best performance which provides $1.1\sim 1.6\times$ improvement against vLLM. In contrast, QLM can only improve the throughput by around $1.04\sim 1.4\times$. The performance improvement of ALPS comes from the hybrid batching to better mix the decode phase with prefill chunks. Application-aware request grouping also contributes to the improvement of throughput because it reduces Time-To-First-Token (TTFT) latency and overall inference time by avoiding redundant computation of shared input parts.

We have noticed that when the batch size equals 32, the performance is significantly better. This is due to the fact that inference performance improves with batch size only up to a threshold, after which bottlenecks arise (memory bandwidth, compute saturation, KV cache pressure).

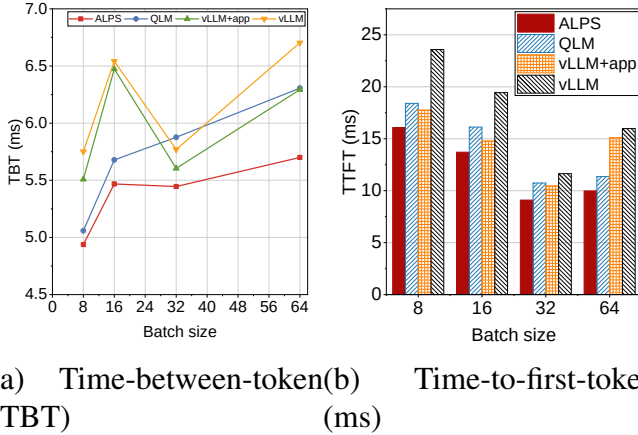


Fig. 8: Time-between token (TBT) and time-to-first-token (TTFT) on a cluster of 3 Gemma-3-1B LLM instances.

TBT and TTFT in Same-Models Cluster. Figure 8a shows the time-between-token for different approaches. ALPS achieves 5-17% speedup compared to vLLM. Although QLM also performs well in some batch sizes, ALPS still receives the best performance by combining request grouping and hybrid batching. QLM and vLLM induce generation stalls by not mixing prefill and decode efficiently, resulting in high latency in TBT. The TBT of vLLM reaches up to 6.5ms and that of QLM reaches up to 6.4ms, while that of ALPS is only around 5.5ms. Figure 8b presents the time-to-first-token for different methods when the batch size ranges from 8 to 64. ALPS provides the lowest TTFT as it groups requests based on applications and maximizes the reuse of prefix caching. This can also be observed from the performance of vLLM+app

approach. QLM and vLLM increase TTFT because they need more time for decoding when a prefix cache is missed. The TTFT of ALPS ranges between 9.1~16.1 ms while that of QLM and vLLM ranges from 10.7~23.6 ms. vLLM+app significantly reduces TTFT over QLM and vLLM, bringing the average TTFT down to the range of 10.4~17.7 ms.

Performance under different request rates.

Figure 9 presents the performance of ALPS compared to baseline schedulers (QLM, vLLM+app, and vLLM) under varying request rates in the Same-Models cluster configuration (three identical Gemma-3-1B models). The request rate was held for 3 seconds for each run. Figure 9a shows the end-to-end (E2E) latency as request rates increase from 25 to 150 requests/s (each rate is maintained for three seconds), where ALPS consistently achieves the lowest latency. This is due to its efficient hybrid batching and prefix reuse. Figure 9b illustrates time-between-token, demonstrating that ALPS maintains an improvement of up to 1.3x across all rates. Figure 9c probably depicts the SLO attainment, with ALPS maintaining the highest values around 95 to 100% because its application-aware grouping eliminates redundant prefill computations and its phase-aware reordering maximizes prefill-decode overlap, even under high request arrival rates where other schedulers suffer from head-of-line blocking and poor batch composition.

Figure 11 shows the cumulative distribution of request latency. ALPS dominates the baselines over most of the distribution, particularly between the 80th and 95th percentiles, indicating consistent improvement rather than gains limited to a small subset of cases. For example, 89% of requests experience less than 2.5 seconds latency, while only 78% of requests for QLM are within 2.5 seconds.

C. Mixed-Models Evaluations

We run the mixed-models evaluations by using three different models (Gemma-3-1b, LLaMa-3-1b and SmolLM2-360M) on a Nvidia Orin cluster with 3 nodes.

Request Latency and Throughput in Mixed-Models Cluster. Figure 10a shows the end-to-end latency and normalized token throughput to vLLM. Over different batch sizes, ALPS provides up to 20% speedup in latency due to several factors: (1) The use of application-aware request grouping reduces repeated computation for the system prompts. (2) The global scheduler prioritizes long-output requests to realize hybrid batching. Although QLM also groups requests and reorder requests, it performs worse in this case because it is unaware of LLM applications and hybrid batching. The E2E latency of ALPS ranges from 1.37~1.5 seconds while that of QLM ranges between 1.46~1.66 seconds. The effect of application-awareness is also reflected in the performance of vLLM+app. Compared to vLLM, vLLM+app reduces the latency by 15%.

As illustrated in Figure 10b, the improvement in token throughput ranges from 12-25% compared to vLLM, while that of QLM only ranges from 2 to 5%. The reason is that efficient hybrid batching can increase the size of token-batches and optimize the GPU utilization. We further notice that

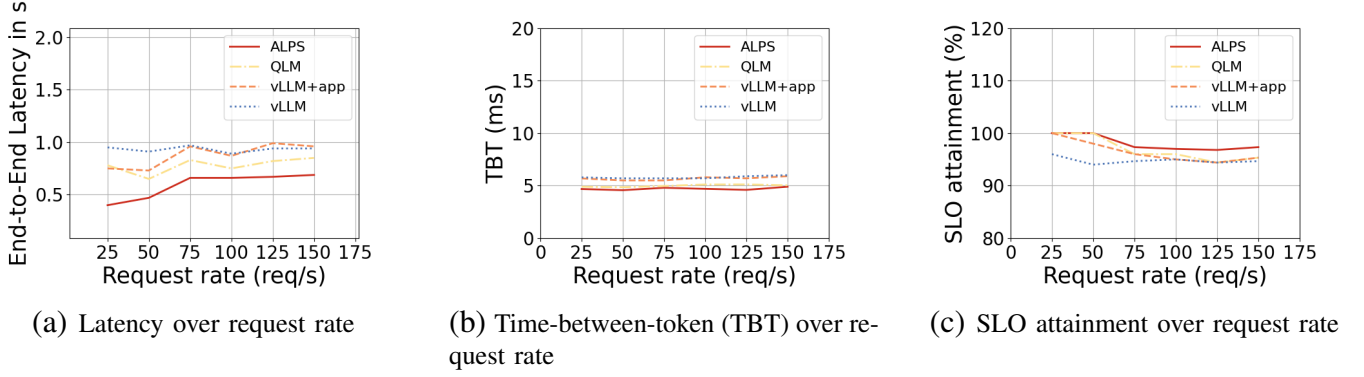
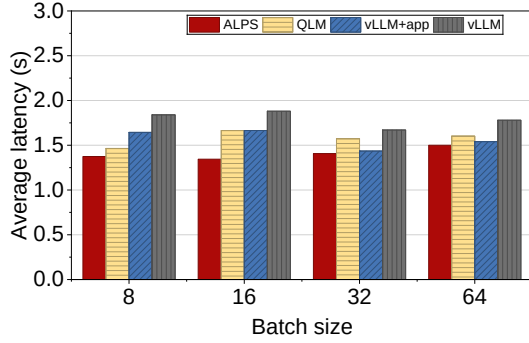
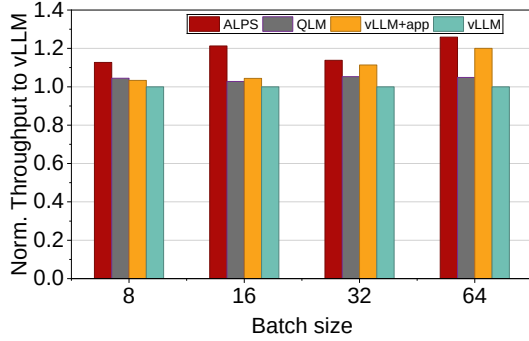


Fig. 9: Performance over different request rates in Same-Models.



(a) E2E Latency in seconds



(b) Token throughput normalized to vLLM

Fig. 10: Performance on a cluster of 3 different LLM instances of LLaMa-3-1B, Gemma-3-1b and SmoLLM2 (Mixed-models).

vLLM+app also improves the token throughput under various settings compared to vLLM. The reason is that the application-aware request grouping reduces the computational overhead and hence reduces the inference time (reusing prefix-caching).

TBT and TTFT in Mixed-Models Cluster. Figure 12a shows the TBT, i.e., time-between-token. We notice that the latency of ALPS fluctuates between 5.5 and 6 ms while that of QLM and vLLM ranges between 6 and 7 ms. We also observe that vLLM+app performs better than vLLM, showing that the application-awareness helps reduce the latency.

Figure 12b shows the TTFT (Time-to-First-Token), which measures the latency of generating the first output token from

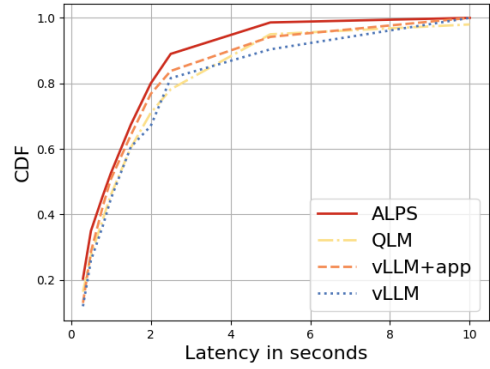


Fig. 11: CDF of latency in a cluster of 3 Nvidia Orin with 3 different models.

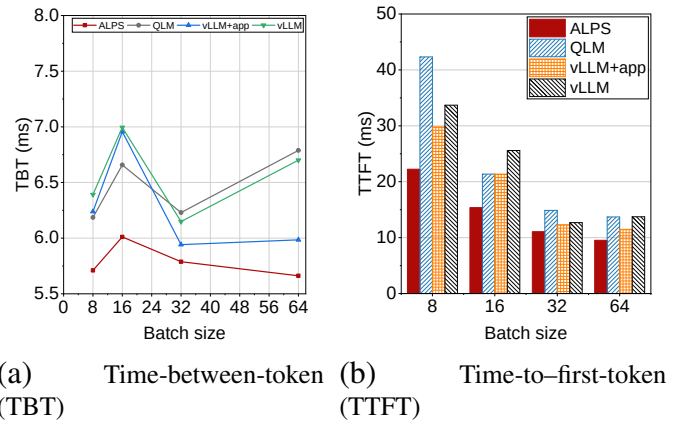


Fig. 12: Performance on a cluster of 3 different LLM instances of LLaMa-3-1B, Gemma-3-1b and SmoLLM2 (Mixed-Models).

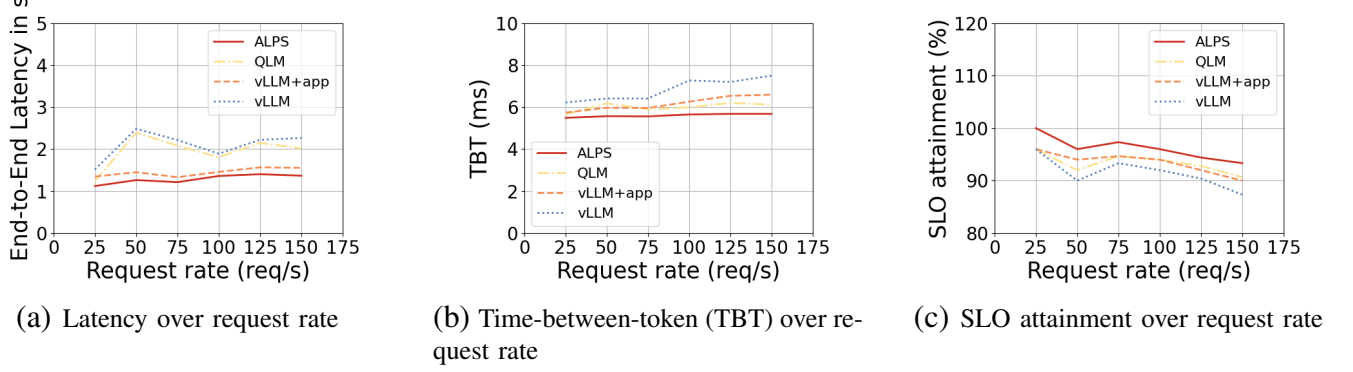


Fig. 13: Performance over different request rates in Mixed-Models.

the time a request arrives in the system. It measures the initial responsiveness of a request, which affects the overall user experience. As shown in Figure 12b, ALPS receives the best performance where the TTFT ranges between 9.52-22.24 ms. In contrast, that of QLM fluctuates between 13.7 and 42.32 ms. This stems from the application-aware grouping, which can also be observed in vLLM+app’s performance. Over different batch sizes, we notice that ALPS always obtains the best performance. This is because our request grouping efficiently reuses the KV cache blocks, so it does not need to recompute from scratch for the system prompts.

Performance against varying request rates in Mixed-Models.

Figure 13 evaluates ALPS under the same varying request rates but in the Mixed-Models cluster, where three different LLMs (Gemma-3-1B, LLaMa-3-1B, and SmoLLM2-360M) are deployed on separate nodes. The request rate was held for 3 seconds for each run. Figure 13a shows that ALPS still achieves the lowest E2E latency in the range between 1.12 and 1.40 seconds, whereas the latency of vLLM and QLM fluctuates between 1.23 and 2.48 seconds. For example, at a request rate of 50, ALPS receives a latency of 1.26 seconds where that of QLM and vLLM are 2.39 and 2.48 seconds, respectively.

Figure 13b displays time-between-token, where ALPS delivers a 12–30% gain over vLLM, whereas QLM only achieves 9–20%, confirming that ALPS’s active hybrid batching (based on generation ratio) is also efficient in mixed-model environments. Figure 13c tracks the SLO attainment, with ALPS again outperforming baselines—SLO attainment ranges from 93 to 100% compared to QLM’s 92 to 96%, because of the global scheduler’s ability to reorder request groups so that decode-heavy requests are prioritized.

D. Sensitivity Analysis

In this subsection, we conduct a sensitivity analysis on different aspects of ALPS. Particularly, we are interested in observing the relationship of prefill and decode time, the impact of varying chunk sizes, SLO violation over batch sizes, OTB prediction accuracy and time overhead of ALPS.

Prefill and Decode Time. Figure 14a shows the prefill time of ALPS over different numbers of requests. We notice that the

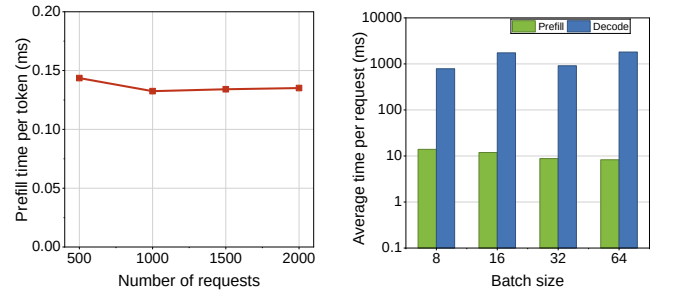


Fig. 14: Prefill and decode time of ALPS with varying number of requests and batch sizes in Gemma-3-1B.

prefill time per token only fluctuates slightly between 0.13 and 0.14 ms. This is because the prefill phase is highly parallel. Figure 14b shows prefill and decode per request over different batch sizes. The decode time is 50-100x higher than prefill time because the decoding process is autoregressive. In other words, decode time dominates the inference time of LLMs.

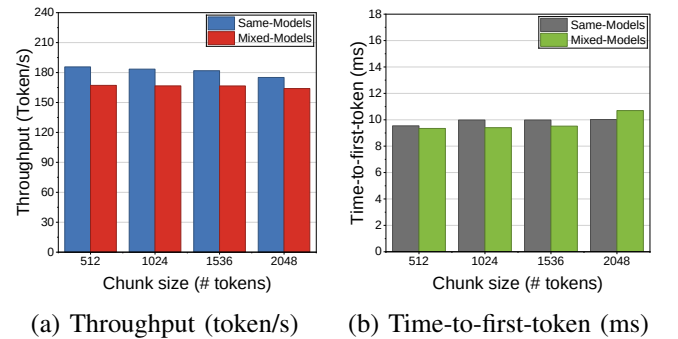
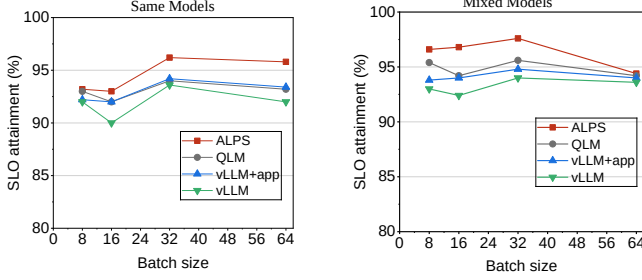


Fig. 15: Throughput and Time-to-first-token of ALPS with varying chunk sizes in Gemma-3-1B.

Throughput and TTFT over Varying Chunk Sizes. Figure 15a shows the token throughput over chunk sizes in [512, 2048]. A larger chunk size means each batch of tokens sent

during prefill contains more tokens. We observe that the token throughput slightly drops with increasing chunk sizes. This is because long chunked prefill finishes prefill phase slowly in the batch and hence damages the effect of hybrid batching. Also, Figure 15b shows that the TTFT increases when the chunk size increases. This stems from the fact that the large chunk sizes lead to long prefill time and hence prolong the TTFT.



(a) Same Models of three Gemma-3-1B instances (b) Mixed models of Gemma-3, Llama-3 and SmolLm instances.

Fig. 16: SLO attainment over varying batch sizes.

SLO Attainment. Figure 16 shows the SLO attainment which is the percentage of requests that successfully meet their predefined latency targets. When the batch size increases from 8 to 64, over 95% requests of ALPS meet the SLO constraint while that of QLM and vLLM ranges from 92-93%. In the same-models and mixed-models settings, the SLO attainment of ALPS only fluctuates between 93 and 97%. The advantages of ALPS stem from efficient prefix sharing and proactive preemption.

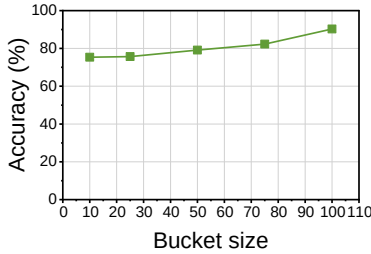


Fig. 17: Prediction accuracy of OTB over different bucket sizes.

OTB Accuracy. Figure 17 reports the prediction accuracy of the OTB predictor as the bucket size varies from 10 to 100. As expected, accuracy improves with larger bucket sizes. Recall that the bucket size corresponds to the length interval into which a request’s output length is classified. Increasing the bucket size reduces the number of categories the predictor must distinguish, thereby improving accuracy. However, excessively large buckets undermine the effectiveness of hybrid batching, as requests within the same group may produce outputs with highly divergent lengths. Conversely, when the bucket size is too small, only a limited number of requests can be aggregated into a group.

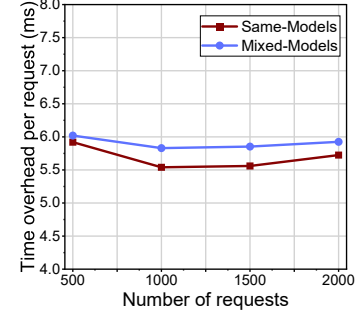


Fig. 18: Time overhead of ALPS.

Time Overhead of ALPS. As shown in Figure 18, the time overhead of ALPS ranges from 5.5 to 6 milliseconds. The time overhead also includes the sequence classifier’s time and decision-making time. It is relatively low as we only predict the range of output tokens, so the number of classifications is significantly lower.

As shown in the previous section, the E2E latency of ALPS is around 1.3-1.5 seconds per request, which means that the time overhead of ALPS is less than 0.5% of the E2E latency per request.

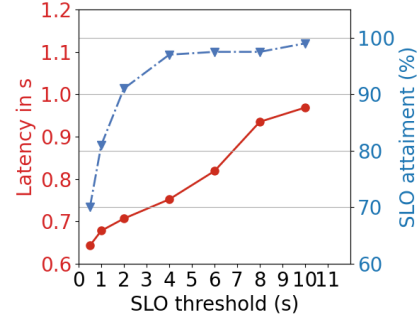


Fig. 19: End-to-end latency and SLO attainment over different SLO thresholds.

Different SLO thresholds. In Figure 19, we evaluate the performance of ALPS under different SLO thresholds ranging from 0.5 to 10 seconds. As the SLO threshold increases from 0.5 to 10 seconds, the end-to-end latency increases from 0.64 to 0.96 s. This is because if the SLO threshold is relaxed, the system can accept higher latency and less preemption will be triggered. Similarly, the SLO attainment rate improves from 70% to 99%. These results demonstrate that ALPS effectively adapts to varying SLO requirements through adaptive request preemption. Under stricter SLO thresholds, ALPS prioritizes requests approaching their deadlines by moving them to the front of the queue.

V. RELATED WORK

Table III summarizes existing works and compares them with ALPS in terms of four aspects, namely, *Multi-instance*, *Application-aware*, *Active hybrid batching*, and *SLO-aware*. Unlike all existing research, ALPS has uniquely addressed

Category	Multi-instance	Application-aware	Active hybrid batching	SLO-aware
Optimizing of LLM backends [9, 12, 13, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57]	✓	✗	✗	✗
Prefix sharing [58, 59, 60]	✓	✗	✗	✓
LLM scheduling [5, 61, 62, 63, 64, 65, 66]	✓	✗	✗	✗
Parrot [18], Ayo [67] and Pie [68]	✓	✓	✗	✗
QLM [34], Past-future [32] and AdaServe [69]	✓	✗	✗	✓
SARATHI [70], SARATHI-Serve [17] and POD-ATTENTION [14]	✗	✗	✓	✓
BatchLLM [71]	✗	✗	✓	✗
PromptCache [72]	✗	✓	✗	✗
ALPS	✓	✓	✓	✓

TABLE III: Comparison of ALPS to related work.

all these aspects. ALPS particularly features two aspects, i.e., application awareness and hybrid batching based on generation ratio—it first identifies the LLM applications and groups them for prefix sharing, and then mixes the chunked prefill phases of new requests with the decode phases of existing requests according to the generation ratio. Next, we discuss related work in multiple categories in detail and highlight how ALPS differs.

LLM Serving Backend. A significant amount of work has focused on improving memory management and token generation by fine-tuning the LLM backends. DiffKV [40], Jenga [49], and DeltaZip [41] propose GPU memory managers to compress fragmented GPU memory to enhance throughput. Menos [42] spatially shares the base model parameters between clients. llm.npu [44] uses a Neural Processing Unit (NPU) on-device to reduce prefill latency. POD-Attention [14] proposes a GPU kernel that carefully allocates GPU’s resources so that prefill and decode operations happen simultaneously. SARATHI [70] uses a single prefill chunk and filling the remaining batch with decodes. Medusa [46] materializes CUDA graphs and KV cache information, while AQUA [47] is a GPU memory management framework that reduces the overhead of paging inference state. Aegaeon [50] uses GPU pooling to perform LLM model auto-scaling at token level. KTransformers [51] and PowerInfer [53] both combine CPU-GPU computing power by using an Expert Deferral mechanism and identifying popular neurons, respectively. LoongServe [52] and NanoFlow [54] both use parallelism to optimize throughput and latency. Serverless-LLM [73] optimizes the checkpointing techniques to speed up LLM inference in serverless computing. Flashgen [43], SpotServe [45], HACK [55], Mell [57] and CacheGen [56] have proposed several techniques to optimize or compress the KV-cache in LLM inference. Jupiter [12] and Spin [13] use pipeline techniques to accelerate LLM inference at the edge. Parrot [18], Ayo [67] and Pie [68] are application-aware, improving system throughput via different techniques such as

Semantic Variable, dataflow graphs and customized service handlers.

ALPS is orthogonal to these existing frameworks that often focus on optimizing the internals of a single LLM backend. Thus, ALPS can be easily integrated with them by deploying ALPS as the gateway for request scheduling for the cluster, where within the cluster these diverse backend techniques can be adopted.

Prefix Sharing. Due to the fact that prefix sharing can significantly reduce the computation in LLM inference, a large amount of work has proposed new architectures for it. Therefore, we use a separate subsection to summarize it.

HotPrefix [74] is a hotness-aware KV cache scheduler that tracks hotness and selectively updates KV caches for LLM inference. ChunkAttention [75] detects matching prefixes across different requests and shares their tensors at runtime for efficiency. SGLang [76] efficiently simplifies programming with primitives for generation and parallelism control, and optimizes KV cache reuse with novel optimizations like Radix-Attention.

The vLLM [16] engine uses PagedAttention to manage the kv-cache blocks of processed requests and reuse these blocks when a new request comes in with the same prefix as previous requests. CacheBlend [58] reuses the precomputed KV caches, regardless prefix or not, and selectively recomputes the KV values of a small subset of tokens to partially update each reused KV cache. InfiniGen [59] proposes a KV-cache system for long-text generation, prefetching only essential entries from KV cache and reducing memory usage. Sarathi-Serve [17] introduces *chunked-prefills*, which splits a prefill request into near equal-sized chunks, improving throughput with large batch sizes. IC-Cache [60] builds a caching system for request-response pairs from larger models as in-context examples, and prepends them as context to new requests’ input. SpecInfer [48] uses small models to predict output tokens and a token tree to create outputs. Both BatchLLM and PromptCache focuses on global prefix caching. Batch-

LLM [71] identifies shared prompts for large industry offline datasets while PromptCache [72] caches text segment for a single-instance scenario.

None of these works identifies the common prefix of system prompts ahead of time for different applications in an online manner and enables explicit reuse. Instead, ALPS manages the prefix reusing explicitly and can reuse the KV context best for the multi-application scenario.

LLM Serving Scheduling. OESD [5] uses auctions to optimize social cost in LLM speculative decoding. Gong *et al.* [32] estimate the peak memory requirement by using the historical distribution of request output lengths, balancing request queuing and harmful evictions. TAPAS [61] uses historical data of temperature and power consumption to place GPU VMs and route requests across LLM instances based on the workloads. ExeGPT [77] optimizes batch size and tensor parallelism, and uses a workload-aware request scheduler to improve system throughput. ThunderServe [62] splits the LLM serving into prefill and decode phases, and uses tabu search and re-scheduling to optimize throughput and latency. Llumnix [63] migrates requests and their state when there is unpredictable workload, resulting in fewer preemptions and interference. LLMShed [64] uses a Bayesian network to profile compound LLM applications and prioritize stages with greater quantified uncertainty reduction in LLM inference. Tabi [65] is optimized for discriminative models and uses a confidence score to decide whether to return the accurate results of small models fast to re-route them to LLMs. A few works [34, 66, 69] schedule requests to optimize SLO for LLM serving. For instance, QLM [34] batches requests with similar SLOs and moves requests that are likely to violate SLOs to the front of the queue by using an ILP solver (Gurobi). AdaServe [69] formulates multi-SLO serving as a constrained problem and uses a speculative tree to meet the latency target.

These works could be the closest to ALPS. Yet, unlike these prior schedulers that treat requests opaquely and often overlook hybrid batching, ALPS uniquely combines application-aware grouping with phase-aware scheduling for edge LLM serving. ALPS proactively groups requests by hashing system prompts to guarantee prefix cache reuse and uses lightweight output prediction to reorder the queue, explicitly optimizing the prefill-decode mix in hybrid batching. This co-design specifically targets edge environments, where both memory efficiency and GPU utilization are paramount.

Overall, ALPS introduces four key novelties as follows.

- 1) Application-aware semantic grouping. ALPS hashes system prompts to group requests from the same application, enabling deterministic prefix-cache reuse before scheduling, rather than relying on opportunistic cache hits during execution.
- 2) Output token prediction. ALPS uses a lightweight token-bucket classifier to predict output length ranges, not exact values. These buckets guide the generation ratio for group ordering, providing sufficient accuracy for phase-aware batching with minimal overhead.
- 3) Phase-aware scheduling. Instead of minimizing per-request completion time, ALPS orders groups by generation ratio to maximize prefill-decode overlap in

hybrid batches, improving GPU utilization and batching efficiency.

- 4) Hierarchical scheduling. ALPS operates at the request-group level, sequentially applying grouping, generation-ratio ordering, and SLO-aware preemption within a unified pipeline that jointly enables prefix sharing and efficient batching.

VI. CONCLUSION AND FUTURE WORK

This paper presents ALPS, a holistic queue scheduler for application-aware LLM inference in edge clusters. It identifies the limitations and the new design space in existing approaches. It explicitly extracts system prompts as a common prefix globally to avoid repeated computation in the prefix cache, and improves the system throughput by prioritizing the requests with larger generation ratios. Requests are scheduled at the level of prefix-sharing groups, and these groups are reordered to achieve the best mix of chunked prefills and decodes in the token batches. Finally, preemption is used for SLO assurance. Evaluations confirm that ALPS outperforms QLM and vLLM substantially in terms of end-to-end latency and throughput on a set of real-world datasets with production LLM models.

While ALPS' phase-mixing optimization is specifically designed for the environments of edge GPUs, its higher-level scheduling intelligence—identifying semantic groups and predicting decode loads—opens compelling future directions. In cloud environments with prefill-decode disaggregation, this intelligence could guide an orchestration layer to route requests efficiently across specialized prefill and decode pools, optimizing for global cache locality and balanced utilization across the entire disaggregated pipeline.

While the current evaluation demonstrates the effectiveness of the proposed approach under the considered experimental settings, larger-scale network-related experiments remain an important direction for future work. In particular, we plan to evaluate the framework in more diverse and realistic distributed environments with varying network topologies, bandwidth constraints, propagation delays, and background traffic conditions. Such experiments will enable a more comprehensive assessment of the system's scalability, robustness, and adaptability under heterogeneous network conditions, while providing deeper insights into its performance in large-scale deployments representative of real-world scenarios.

REFERENCES

- [1] Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanchao Shu, and Jiming Chen. A review on edge large language models: Design, execution, and applications. *ACM Comput. Surv.*, 57(8), March 2025. ISSN 0360-0300. doi: 10.1145/3719664. URL <https://doi.org/10.1145/3719664>.
- [2] Miao Hu, Qi He, and Di Wu. Qllms: Quantization-adaptive llm scheduling for partially informed edge serving systems. In *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, pages 1–10, 2025.
- [3] Shengyuan Ye, Bei Ouyang, Tianyi Qian, Liekang Zeng, Jingyi Li, Jiangsu Du, Xiaowen Chu, Guoliang Xing, and Xu Chen. Resource-efficient personal large language models fine-tuning with collaborative edge computing. *IEEE Transactions on Parallel and Distributed Systems*, 37(3):680–696, 2026. doi: 10.1109/TPDS.2026.3654957.
- [4] Kaiyuan Liu, Xiaobo Zhou, and Li Li. m²llm: A multi-dimensional optimization framework for llm inference on mobile devices. *IEEE*

- Transactions on Parallel and Distributed Systems*, 36(10):2014–2029, 2025. doi: 10.1109/TPDS.2025.3587445.
- [5] Mingtao Ji, Lei Jiao, Bin Tang, Zhihao Qu, Chen Chen, and Baoliu Ye. Incentivizing and orchestrating cloud-edge llm speculative decoding via auctions. In *2026 IEEE 46th International Conference on Distributed Computing Systems (ICDCS)*, pages 425–435, 2026. doi: 10.1109/2575-8411.2026.00047.
 - [6] Xiangchen Li, Dimitrios Spatharakis, Saeid Ghafouri, Jiakun Fan, Hans Vandierendonck, Deepu John, Bo Ji, and Dimitrios S. Nikolopoulos. Sled: A speculative llm decoding framework for efficient edge serving. In *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing, SEC '25*, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400722387. doi: 10.1145/3769102.3770608. URL <https://doi.org/10.1145/3769102.3770608>.
 - [7] Erik Johannes Husom, Arda Goknil, Merve Asteekin, Lwin Khin Shar, Andre KÅsen, Sagar Sen, Benedikt Andreas Mithassel, and Ahmet Soylu. Sustainable llm inference for edge ai: Evaluating quantized llms for energy efficiency, output accuracy, and inference latency. *ACM Trans. Internet Things*, 6(4), November 2025. doi: 10.1145/3767742. URL <https://doi.org/10.1145/3767742>.
 - [8] Yufeng Gu, Alireza Khadem, Sumanth Umesh, Ning Liang, Xavier Servot, Onur Mutlu, Ravi Iyer, and Reetuparna Das. *PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference*, page 862–881. Association for Computing Machinery, New York, NY, USA, 2025.
 - [9] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, Santa Clara, CA, July 2024. USENIX Association.
 - [10] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132, 2024.
 - [11] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Inference without interference: Disaggregate llm inference for mixed downstream workloads, 2024. URL <https://arxiv.org/abs/2401.11181>.
 - [12] Shengyuan Ye, Bei Ouyang, Liekang Zeng, Tianyi Qian, Xiaowen Chu, Jian Tang, and Xu Chen. Jupiter: Fast and resource-efficient collaborative inference of generative llms on edge devices. In *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, pages 1–10, 2025.
 - [13] Fahao Chen, Peng Li, Tom H. Luan, Zhou Su, and Jing Deng. Spin: Accelerating large language model inference with heterogeneous speculative models. In *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, pages 1–10, 2025.
 - [14] Aditya K. Kamath, Ramya Prabhu, Jayashree Mohan, Simon Peter, Ramachandran Ramjee, and Ashish Panwar. Pod-attention: Unlocking full prefill-decode overlap for faster llm inference. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, page 897–912, 2025.
 - [15] Arindam Mitra, Hamed Khanpour, Corby Rosset, and Ahmed Awadallah. Orca-math: Unlocking the potential of slms in grade school math, 2024. URL <https://arxiv.org/abs/2402.14830>.
 - [16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, page 611–626, New York, NY, USA, 2023. Association for Computing Machinery.
 - [17] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 117–134, Santa Clara, CA, July 2024. USENIX Association.
 - [18] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 929–945, July 2024.
 - [19] Jiazhi Jiang, Yao Chen, Zining Zhang, Bingsheng He, Pingyi Luo, Mian Lu, Yuqiang Chen, Hongbing Zhang, Jiangsu Du, Dan Huang, and Yutong Lu. Efficient kv cache spillover management on memory-constrained gpu for llm inference. *IEEE Transactions on Parallel and Distributed Systems*, 37(1):90–105, 2026. doi: 10.1109/TPDS.2025.3626974.
 - [20] A collection of gpt system prompts and various prompt injection/leaking knowledge., 2025. URL https://github.com/LouisShark/chatgpt_system_prompt/tree/main.
 - [21] llm d. a kubernetes-native high-performance distributed llm inference framework, 2025. URL <https://llm-d.ai/>. <https://llm-d.ai/>.
 - [22] openai/tiktoken: tiktoken is a fast bpe tokeniser for use with openai's models., 2025. URL <https://github.com/openai/tiktoken>.
 - [23] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024.
 - [24] Hugging Face. Quixai/systemchat-2.0, 8 2024. URL <https://huggingface.co/datasets/QuixiAI/SystemChat-2.0>.
 - [25] Metagpt: The multi-agent framework: First ai software company, towards natural language programming, 2025. URL <https://github.com/FoundationAgents/MetaGPT>.
 - [26] Microsoft copilot: Your ai companion, 2025. URL <https://copilot.micr osoft.com/>.
 - [27] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA, July 2022. USENIX Association.
 - [28] Apache kafka, 2025. URL <https://kafka.apache.org/>.
 - [29] Openai-compatible server - vllm, 2025. URL https://docs.vllm.ai/en/latest/serving/openai_compatible_server/#score-api.
 - [30] distilbert/distilbert-base-uncased · hugging face, 3 2024. URL <https://huggingface.co/distilbert/distilbert-base-uncased>. <https://huggingface.co/distilbert/distilbert-base-uncased>.
 - [31] Bert, 2025. URL https://huggingface.co/docs/transformers/en/model_doc/bert. https://huggingface.co/docs/transformers/en/model_doc/bert.
 - [32] Ruihao Gong, Shihao Bai, Siyu Wu, Yunqian Fan, Zaijun Wang, Xiuhong Li, Hailong Yang, and Xianglong Liu. Past-future scheduler for llm serving under sla guarantees. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '25*, page 798–813, 2025.
 - [33] esnet. iperf3: A tcp, udp, and sctp network bandwidth measurement tool · github, 2026. URL <https://github.com/esnet/iperf>. <https://github.com/esnet/iperf>.
 - [34] Archit Patke, Dharmath Reddy, Saurabh Jha, Haoran Qiu, Christian Pinto, Chandra Narayanaswami, Zbigniew Kalbarczyk, and Ravishanker Iyer. Queue management for slo-oriented large language model serving. In *Proceedings of the 2024 ACM Symposium on Cloud Computing, SoCC '24*, page 18–35, 2024.
 - [35] Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B Hashimoto. Length-controlled alpacaEval: A simple way to debias automatic evaluators. *arXiv preprint arXiv:2404.04475*, 2024.
 - [36] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. MS MARCO: A human-generated MACHine reading COMprehension dataset, 2017. URL <https://openreview.net/forum?id=Hk1iOLcle>.
 - [37] Jiacheng Ye, Chengzu Li, Lingpeng Kong, and Tao Yu. Generating data for symbolic language with large language models, 2023. URL <https://arxiv.org/abs/2305.13917>.
 - [38] Finance-Alpaca. gbbarti/finance-alpaca · datasets at hugging face, 12 2024. URL <https://huggingface.co/datasets/gbbarti/finance-alpaca>.
 - [39] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
 - [40] Yanqi Zhang, Yuwei Hu, Runyuan Zhao, John C. S. Lui, and Haibo Chen. Diffkv: Differentiated memory management for large language models with parallel kv compaction. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP '25*, page 431–445, 2025.
 - [41] Xiaozhe Yao, Qinghao Hu, and Ana Klimovic. Deltazip: Efficient serving of multiple full-model-tuned llms. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys '25*, page 110–127, New York, NY, USA, 2025. Association for Computing

- Machinery.
- [42] Chenghao Hu and Baochun Li. Menos: Split fine-tuning large language models with efficient gpu memory sharing. In *Proceedings of the 25th International Middleware Conference*, Middleware '24, page 185–198, New York, NY, USA, 2024. Association for Computing Machinery.
 - [43] Jinwoo Jeong and Jeongseob Ahn. Accelerating llm serving for multi-turn dialogues with efficient resource management. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, page 1–15, New York, NY, USA, 2025. Association for Computing Machinery.
 - [44] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. Fast on-device llm inference with npus. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS '25, page 445–462, New York, NY, USA, 2025. Association for Computing Machinery.
 - [45] Xupeng Miao, Chunan Shi, Jiangfei Duan, Xiaoli Xi, Dahua Lin, Bin Cui, and Zhihao Jia. Spotserv: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS '24, page 1112–1127, New York, NY, USA, 2024. Association for Computing Machinery.
 - [46] Shaoxun Zeng, Minhui Xie, Shiwei Gao, Youmin Chen, and Youyou Lu. Medusa: Accelerating serverless llm inference with materialization. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS '25, page 653–668, New York, NY, USA, 2025. Association for Computing Machinery.
 - [47] Abhishek Vijaya Kumar, Gianni Antichi, and Rachee Singh. *Aqua: Network-Accelerated Memory Offloading for LLMs in Scale-Up GPU Domains*, page 48–62. Association for Computing Machinery, New York, NY, USA, 2025.
 - [48] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, page 932–949, New York, NY, USA, 2024. Association for Computing Machinery.
 - [49] Chen Zhang, Kuntai Du, Shu Liu, Woosuk Kwon, Xiangxi Mo, Yufeng Wang, Xiaoxuan Liu, Kaichao You, Zhuohan Li, Mingsheng Long, Jidong Zhai, Joseph Gonzalez, and Ion Stoica. Jenga: Effective memory management for serving llm with heterogeneity. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, SOSP '25, page 446–461, New York, NY, USA, 2025. Association for Computing Machinery.
 - [50] Yuxing Xiang, Xue Li, Kun Qian, Yufan Yang, Diwen Zhu, Wenyuan Yu, Ennan Zhai, Xuanzhe Liu, Xin Jin, and Jingren Zhou. Aegaeon: Effective gpu pooling for concurrent llm serving on the market. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, SOSP '25, page 1030–1045, New York, NY, USA, 2025. Association for Computing Machinery.
 - [51] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. Ktransformers: Unleashing the full potential of cpu/gpu hybrid inference for moe models. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, SOSP '25, page 1014–1029. Association for Computing Machinery, 2025.
 - [52] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, SOSP '24, page 640–654, 2024.
 - [53] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, SOSP '24, page 590–606, 2024.
 - [54] Kan Zhu, Yufei Gao, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, et al. {NanoFlow}: Towards optimal large language model serving throughput. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 749–765, 2025.
 - [55] Zeyu Zhang, Haiying Shen, Shay Vargaftik, Ran Ben Basat, Michael Mitzenmacher, and Minlan Yu. Hack: Homomorphic acceleration via compression of the key-value cache for disaggregated llm inference. In *Proceedings of the ACM SIGCOMM 2025 Conference*, SIGCOMM '25, page 1245–1247, 2025.
 - [56] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 38–56, New York, NY, USA, 2024. Association for Computing Machinery.
 - [57] Qianli Liu, Zicong Hong, Peng Li, Fahao Chen, and Song Guo. Mell: Memory-efficient large language model serving via multi-gpu kv cache management. In *IEEE Conference on Computer Communications*, pages 1–10, 2025.
 - [58] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys '25, page 94–109, New York, NY, USA, 2025. Association for Computing Machinery.
 - [59] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 155–172, July 2024.
 - [60] Yifan Yu, Yu Gan, Nikhil Sarda, Lillian Tsai, Jiaming Shen, Yanqi Zhou, Arvind Krishnamurthy, Fan Lai, Hank Levy, and David Culler. Ic-cache: Efficient large language model serving via in-context caching. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, SOSP '25, page 375–398, New York, NY, USA, 2025. Association for Computing Machinery.
 - [61] Jovan Stojkovic, Chaojie Zhang, Inigo Goiri, Esha Choukse, Haoran Qiu, Rodrigo Fonseca, Josep Torrellas, and Ricardo Bianchini. *TAPAS: Thermal- and Power-Aware Scheduling for LLM Inference in Cloud Platforms*, page 1266–1281. Association for Computing Machinery, New York, NY, USA, 2025.
 - [62] Youhe Jiang, Fangcheng Fu, Xiaozhe Yao, Taiyi Wang, Bin Cui, Ana Klimovic, and Eiko Yoneki. Thunderserve: High-performance and cost-efficient llm serving in cloud environments. <https://arxiv.org/abs/2502.09334>, 2025.
 - [63] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 173–191, Santa Clara, CA, July 2024. USENIX Association.
 - [64] Botao Zhu, Chen Chen, Xiaoyi Fan, and Yifei Zhu. Llmsched: Uncertainty-aware workload scheduling for compound llm applications. In *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDCS)*, pages 527–537, 2025.
 - [65] Yiding Wang, Kai Chen, Haisheng Tan, and Kun Guo. Tabi: An efficient multi-level inference system for large language models. In *Proceedings of the Eighteenth European Conference on Computer Systems*, EuroSys '23, page 233–248, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394871. doi: 10.1145/3552326.3587438. URL <https://doi.org/10.1145/3552326.3587438>.
 - [66] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 965–988, Santa Clara, CA, July 2024. USENIX Association.
 - [67] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. Towards end-to-end optimization of llm-based applications with ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, page 1302–1316, New York, NY, USA, 2025. Association for Computing Machinery.
 - [68] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. Pie: A programmable serving system for emerging llm applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, SOSP '25, page 415–430, New York, NY, USA, 2025. Association for Computing Machinery.
 - [69] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, Zeyu Wang, Qinghan Chen, Shuhuai Lin, April Yang, Zhihao Zhang, Zhuoming Chen, Sean Lai, Xinhao Cheng, Xupeng Miao, and Zhihao Jia. Adaserve: Accelerating multi-slo llm serving with slo-customized spec-

- ulative decoding, 2025. URL <https://arxiv.org/abs/2501.12162>.
- [70] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills, 2023. URL <https://arxiv.org/abs/2308.16369>.
- [71] Zhen Zheng, Xin Ji, Taosong Fang, Fanghao Zhou, Chuanjie Liu, and Gang Peng. Batchllm: Optimizing large batched llm inference with global prefix sharing and throughput-oriented token batching, 2025. URL <https://arxiv.org/abs/2412.03594>.
- [72] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In P. Gibbons, G. Pekhimenko, and C. De Sa, editors, *Proceedings of Machine Learning and Systems*, volume 6, pages 325–338, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/file/a66caa1703fe34705a4368c3014c1966-Paper-Conference.pdf.
- [73] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. ServerlessLLM: Low-Latency serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 135–153, July 2024.
- [74] Yuhang Li, Rong Gu, Chengying Huan, Zhibin Wang, Renjie Yao, Chen Tian, and Guihai Chen. Hotprefix: Hotness-aware kv cache scheduling for efficient prefix sharing in llm inference systems. *Proc. ACM Manag. Data*, 3(4), September 2025. doi: 10.1145/3749168.
- [75] Lu Ye, Ze Tao, Yong Huang, and Yang Li. ChunkAttention: Efficient self-attention with prefix-aware KV cache and two-phase partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 11608–11620, August 2024.
- [76] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- [77] Hyungjun Oh, Kihong Kim, Jaemin Kim, Sungkyun Kim, Junyeol Lee, Du-seong Chang, and Jiwon Seo. Exegpt: Constraint-aware resource scheduling for llm inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS '24*, page 369–384, New York, NY, USA, 2024. Association for Computing Machinery.



Richard Mortier is Professor of Computing & Human-Data interaction at Cambridge University and a Fellow of Christ's College. He is a computer scientist whose research has included distributed system performance monitoring and debugging, incentives in Internet routing protocols, real-time media platform design and implementation, and platforms for privacy preserving personal data processing. Current research includes computing at the edge of the network, GPU and CPU scheduling, energy efficiency, and personal distributed computing. Prior to rejoining the University, he spent time at the University of Nottingham, Microsoft Research Cambridge, Sprint Advanced Technology Lab (USA), and founding startups.



Chen Chen is an Assistant Professor at Nottingham Trent University, UK and a senior member of St Edmund's College, Cambridge. He was a postdoctoral researcher in the System Research Group, Department of Computer Science and Technology, University of Cambridge. His research interests are broadly in areas of Systems for AI/LLM, Serverless Computing, Cloud/Edge Computing, Network Resource Orchestration, and Distributed Systems.



Lei Jiao received the Ph.D. degree from the University of Göttingen, Germany in 2014. He is currently a faculty member at the University of Oregon, USA and was previously a technical staff member at Nokia Bell Labs, Ireland. He researches machine learning systems, edge computing, cloud computing, network optimization, and network economics. He has published over 100 papers in journals such as JSAC, ToN, TPDS, TMC, and TSC and conferences such as INFOCOM, MOBIHOC, ICNP, ICDCS, SECON, and IPDPS. He is a recipient of the NSF CAREER award. He also received the Best Paper Awards of IEEE LANMAN 2013 and IEEE CNS 2019, as well as the 2016 Alcatel-Lucent Bell Labs UK and Ireland Recognition Award. He has been on the program committees of many conferences including INFOCOM, MOBIHOC, ICDCS, and IWQoS.